
FROM AGENT LOOPS TO DETERMINISTIC GRAPHS: EXECUTION LINEAGE FOR REPRODUCIBLE AI-NATIVE WORK

A PREPRINT

Josh Rosen
ThruWire, Inc.

Seth Rosen
ThruWire, Inc.

May 4, 2026

ABSTRACT

Large language model (LLM) systems are increasingly deployed as agentic workflows that interleave reasoning, tool use, and iterative refinement. While effective for task completion, these systems rely on implicit conversational state and stochastic control loops, limiting reproducibility, inspectability, and incremental recomputation.

We introduce an execution model that replaces agent loops with explicit directed acyclic graphs (DAGs) of computation, where nodes produce structured intermediate artifacts and declare dependencies. We define execution lineage as the explicit representation of computation structure, contrasting with prompt lineage in existing systems. Under fixed inputs and configurations, execution proceeds deterministically as a forward pass, enabling replay, partial recomputation, and consistent propagation of changes.

We formalize the model, situate it within prior work on agent systems and workflow engines, and present an empirical comparison showing reduced variance (6.2x), lower recomputation cost (3.8x), and improved traceability relative to loop-based approaches. Our central claim is that the bottleneck in many AI-native systems is no longer model capability alone, but the absence of durable computational structure outside the model. We argue that execution lineage is a necessary abstraction for scaling AI systems beyond ephemeral task completion toward persistent, collaborative workflows in which humans and agents iteratively build on shared intermediate state.

1 Introduction

Agent-based LLM systems have emerged as a dominant paradigm for applying large language models to complex tasks. Approaches such as ReAct [9] combine reasoning and action within iterative loops, enabling dynamic tool usage and adaptive planning.

This paradigm has been productive because it matches the interface of current language models: a prompt is assembled, the model responds, tools may be invoked, and the loop repeats until an answer appears satisfactory. In short-horizon tasks this is often sufficient. A user can tolerate some variance, intermediate reasoning can remain transient, and restarting the loop is usually cheaper than building explicit structure.

In practice, such systems rarely expose the raw model alone. They run inside an *agent harness*: a surrounding control layer that assembles prompts, manages tools and memory, and decides when execution should continue, branch, or terminate. Recent research has started to formalize the harness itself as an object of study rather than a hidden implementation detail [21, 22]. This shift is important because many system behaviors traditionally attributed to the model are in fact produced by the harness.

The problem changes once the work becomes multi-step, stateful, and long-lived. In those settings, the system is no longer producing a single answer in one conversational thread. It is generating intermediate research, analyses, transformations, and reports that may be revised later by humans or reused by other agents. What matters is not only whether the final answer is plausible, but whether the system can explain what it depended on, selectively recompute what changed, and preserve stable boundaries between stages of work.

Despite their flexibility, these systems exhibit systemic limitations:

- **Non-deterministic execution:** identical inputs may produce different outputs.
- **Implicit dependencies:** execution structure is embedded in conversational history.
- **Limited reproducibility:** intermediate reasoning is not preserved in structured form.
- **Global recomputation:** small changes require full re-execution.

These properties are acceptable for single-task workflows but break down for long-lived systems where consistency and incremental updates are required.

We propose a shift from **prompt lineage**—where state is encoded in evolving prompts—to **execution lineage**, where computation is represented explicitly as a graph.

The distinction is architectural. Prompt lineage records how prompts and transcripts evolved over time, but it does not provide a stable substrate on which downstream computation can depend. Execution lineage instead externalizes the structure of the work itself: each unit of work declares its inputs, its dependencies, its output contract, and its execution identity. Under this model, intermediate artifacts are first-class objects rather than incidental text inside a prompt window.

Our research is motivated by a practical observation: many failures attributed to “LLM unreliability” are in fact failures of system structure. If upstream results are passed around as ad hoc context, then reuse is heuristic, inspection is manual, and re-running a workflow means reconstructing state through prompts. If the same work is represented as a graph of explicit execution units with stable intermediate boundaries, then replay, observability, and partial recomputation become system properties rather than prompt engineering tricks.

This paper makes three contributions. First, we articulate execution lineage as a computational abstraction for AI-native workflows. Second, we formalize a deterministic DAG-based execution model that separates authored structure from model-time generation. Third, we provide an empirical comparison against a loop-based baseline on a representative analysis workflow, showing lower variance, cheaper recomputation, and materially better traceability. More broadly, we argue that the trajectory of AI systems increasingly resembles earlier transitions in data engineering and build systems: as workflows become collaborative and persistent, implicit orchestration must give way to explicit dependency structure.

The paper’s scope is intentionally narrow. Our primary concern is *how* AI-native work executes: what the unit of recomputation is, what makes a prior result replayable, and how dependency changes propagate. Richer questions about the semantics of intermediate artifacts or about multi-party co-authoring are important, but they are orthogonal to the execution-lineage thesis developed here.

2 Related Work

2.1 Surveys and Taxonomies

The literature on LLM agents has already grown large enough to generate several dedicated surveys. Recent reviews cover planning [1], memory [3], broader workflow paradigms spanning tool use, planning, and feedback learning [2], and, more recently, memory mechanisms and evaluation through early 2026 [4]. This matters for our argument because it shows that the field has largely converged on the agent as a composite system rather than a raw model invocation. The open question is no longer whether external tools, memory, and iteration matter, but how they should be organized into an execution substrate that remains legible as tasks become longer-horizon and more stateful.

Most survey taxonomies still group systems by capabilities such as planning, reflection, memory, and tool use. Our taxonomy cuts across those dimensions. We focus instead on whether the substrate of execution is implicit or explicit: whether intermediate work remains embedded in prompts and transcripts, or whether it is exposed as stable computational structure with declared dependencies and replayable boundaries.

2.2 Agentic LLM Systems

ReAct [9] introduced interleaved reasoning and action. Toolformer [10] trains models to use tools via self-supervision. A parallel line of work studies agent frameworks and multi-agent organizations, including CAMEL [11], AutoGen [14], MetaGPT [13], and Voyager [12]. These systems show that substantial capability gains can be achieved by placing the model inside richer interaction loops, role structures, and tool environments.

These approaches share a control-loop architecture:

$$s_{t+1} = f(s_t, \text{LLM}, \text{tools}) \quad (1)$$

where state is implicit.

The strength of this family of systems is adaptability. The loop can decide at runtime which tool to call, what sub-problem to attack next, or how to revise the prompt after observing a failure. This makes agentic systems effective at open-ended problem solving, especially when the horizon is short and the cost of revisiting prior work is low.

However, the same flexibility creates structural ambiguity. Dependencies are often encoded in conversational state rather than in explicit program structure. Intermediate results may exist only as prompt fragments or narration. Two executions that are semantically “the same” can differ because context was assembled differently, tool results were reformatted, or reasoning unfolded in another order. As a consequence, the system can appear capable while remaining difficult to inspect, replay, or maintain.

Recent work on planning agents, including Tree-of-Thoughts [15], improves search over reasoning trajectories but retains this basic property: the reasoning process is made more elaborate without necessarily becoming an explicit reusable execution substrate. The same is true of many role-based and multi-agent frameworks. They improve decomposition, coordination, and performance, but usually treat execution state as something reconstructed through conversation rather than compiled and persisted as dependency structure. In our framing, these methods improve inference *inside* a run, whereas execution lineage addresses structure *across* runs.

2.3 Agent Harnesses and Runtimes

The term *agent harness* has recently gained research traction as a way to name the surrounding machinery that makes an LLM agent operational. Natural-Language Agent Harnesses [21] explicitly studies harness engineering as a portable object, arguing that important agent behavior lives in editable control logic rather than only in model weights or prompts. General Modular Harness for LLM Agents in Multi-Turn Gaming Environments [22] similarly studies harness design as a composition of modules such as perception, memory, and reasoning.

This line of work is closely related to ours because it externalizes the wrapper around the model. Memory-oriented work pushes in the same direction. Agent Workflow Memory [23] learns reusable workflows from prior trajectories, while On the Structural Memory of LLM Agents [24] studies alternative memory structures and retrieval schemes. WorkflowLLM [25] goes further by treating workflow orchestration itself as a capability to be trained and benchmarked. Recent 2025–2026 work pushes farther toward persistent procedural state: LEGOMem [26] studies modular procedural memory for workflow automation, Memory-R1 [27] studies learned memory management policies, and Memori [28] argues that persistent memory should be treated as a structured systems layer rather than as raw conversational carry-forward. These papers all suggest that the structure around the model is an important locus of capability.

Our contribution differs in emphasis. Harness and memory papers ask how agent behavior can be encoded, adapted, or transferred at the controller level. We ask what execution model the harness should implement when workflows need deterministic replay, explicit lineage, and partial recomputation. In that sense, execution lineage can be understood as a systems-level proposal for what a rigorous harness runtime should optimize around. It is compatible with memory-augmented agents, but it is not reducible to memory. A memory system decides what to retain and retrieve; an execution-lineage system decides what the unit of computation is, what its dependencies are, and what exactly must be rerun when some upstream state changes.

2.4 Reasoning Traces, Search, and Intermediate Steps

Work on chain-of-thought and related prompting methods establishes an important empirical premise for our paper: intermediate reasoning steps often improve final-task performance. Chain-of-Thought prompting [6] shows that exposing intermediate reasoning traces can materially improve complex reasoning. Self-Consistency [7] improves on greedy chain-of-thought by sampling multiple reasoning paths and selecting the most consistent answer. Least-to-Most prompting [8] demonstrates gains from decomposing harder problems into simpler sequential subproblems. Show Your Work: Scratchpads for Intermediate Computation with Language Models [5] similarly shows that intermediate computation traces can substantially improve multi-step tasks.

Search-based methods extend this idea further. Tree-of-Thoughts [15] treats intermediate thoughts as expandable search nodes rather than a single left-to-right trace. Language Agent Tree Search [18] combines reasoning, acting, and planning through search over agent trajectories. Reflexion [16], Self-Refine [17], and A Zero-Shot Language Agent for Computer Control with Structured Reflection [19] show that iterative critique, reflection, and repair can improve downstream results across diverse tasks. Earlier work on workflow-guided exploration in web environments [20] also foreshadows the importance of reusable action structure, albeit in a pre-LLM reinforcement learning setting.

These papers are important to our argument for two reasons. First, they provide evidence that intermediate steps are not incidental; they are often where performance gains come from. Second, they also reveal the limitation of prompt-centric intermediate state: most of these techniques still represent intermediate work as textual traces tied to a specific execution episode. The system may benefit from them during inference, yet still lack stable, replayable boundaries between computational units. Our proposal builds on the empirical value of intermediate reasoning while relocating its substrate from transient text into explicit execution structure.

2.5 Workflow and DAG Systems

Workflow and dataflow systems such as Dryad [29] and Spark [30], as well as orchestration and analytics systems such as Airflow [31] and dbt [32], model execution around explicit dependency graphs, scheduled runs, and lineage-aware transformations.

These systems emphasize:

- explicit dependencies
- deterministic execution
- incremental recomputation

The relevance of these systems is not superficial. They reflect a mature systems answer to a recurring problem: when work becomes multi-stage and collaborative, hidden dependencies and ephemeral outputs do not scale. Data engineering historically moved from scripts and manual handoffs toward compiled graphs, materialized artifacts, and lineage-aware recomputation because these primitives were necessary for correctness and maintainability. We view AI-native workflows as reaching a similar inflection point.

That said, classical DAG systems were not designed around stochastic model calls, evolving prompts, or semantically rich intermediate artifacts. Their nodes typically represent deterministic code transformations with well-defined data contracts. Our work therefore does not merely import DAGs into AI systems; it adapts the DAG abstraction to settings where nodes may invoke LLMs, generate structured artifacts, and depend on both model configuration and upstream execution identity.

2.6 Programmatic LLM Systems

Emerging work treats LLMs as programs:

- DSPy [33]
- LMQL [34]
- PAL [36]
- Program of Thoughts [35]

These frameworks introduce valuable structure around prompting, constraints, optimization, and output shaping. They make it easier to define reusable modules, specify formats, and reason about prompt behavior more systematically than ad hoc chat loops allow.

Nevertheless, most remain primarily sequential or call-graph oriented rather than execution-lineage oriented. They improve how a single pipeline is programmed, but they generally do not make dependency materialization, replay identity, or partial downstream invalidation the organizing abstraction of the whole system. In other words, they help program model interactions, while our focus is on how a larger AI workflow is represented, executed, and persisted over time.

2.7 Benchmarks and Reproducible Agent Environments

Another relevant literature studies how to evaluate agents in realistic yet reproducible environments. AgentBench [38], WebArena [39], VisualWebArena [40], and WorkArena [41] evaluate web-based and enterprise-style tasks. AndroidWorld [42], OSWorld [43], and AppWorld [44] extend this trend toward richer execution environments with state-based or execution-based evaluation. GAIA [45] evaluates assistants on realistic tool-using tasks, while LifelongAgentBench [46] targets long-term accumulation and transfer across interdependent tasks.

Recent 2025–2026 benchmark work sharpens the memory and persistence angle in particular. MemoryAgentBench [47], Evo-Memory [48], and Mem2ActBench [49] all argue that static one-shot evaluation misses the harder problem

of incremental accumulation, selective forgetting, and task-conditioned reuse. These benchmarks are not execution-lineage systems, but they are strong evidence that the field is moving toward persistent, longitudinal evaluation rather than isolated agent episodes.

This benchmark literature is important because it increasingly treats *environmental reproducibility* and *execution-based evaluation* as first-class concerns. However, benchmark reproducibility is not the same as workflow reproducibility. A benchmark may make the external environment resettable and measurable while still leaving the internal execution structure of the agent implicit. Our work is complementary: we focus on making the agent’s own computational lineage explicit and incrementally reusable.

2.8 Workflow Provenance and Interactive Analysis

There is also an emerging line of work on provenance-aware agent systems in scientific and workflow settings. LLM Agents for Interactive Workflow Provenance [50] studies how LLM agents can query and analyze workflow provenance records through modular interfaces. Connecting Large Language Model Agent to High Performance Computing Resource [51] examines how tool-using agents can operate over parallel and distributed execution substrates.

These papers are adjacent to our thesis because they treat workflows and provenance as serious systems objects rather than as informal prompt context. The difference is that provenance work typically assumes an existing workflow system whose traces are queried after the fact. Execution lineage, as we define it, moves provenance into the execution substrate itself: the lineage is not only something to inspect later, but part of what determines identity, replay, and invalidation during execution.

2.9 Reproducibility and Evaluation

LLM reproducibility challenges are well documented [37]. Variance arises from sampling, model updates, and tool interactions.

Prior work has correctly emphasized that reproducibility in language model systems is limited by multiple factors, including decoding stochasticity, external API variation, and infrastructure drift. Work on interactive evaluation [52] and realistic agent benchmarks [42, 43] likewise shows that even controlled environments exhibit sensitivity to rollout details and test conditions. We do not claim to eliminate all sources of variance. Instead, we isolate a source that is architectural and tractable: execution structure itself. Even when model and tool behavior are held fixed, prompt-oriented systems often reconstitute state heuristically at runtime. This introduces unnecessary instability and obscures provenance.

Our work complements reproducibility research by shifting attention from model behavior alone to the execution substrate around the model. The question is not only whether a model can reproduce a token sequence, but whether the system can reproduce a computational step, restore its exact intermediate state, and determine precisely which downstream work should be invalidated when an upstream artifact changes.

3 Execution Lineage Model

We define a workflow as:

$$G = (V, E) \quad (2)$$

where V is a set of executable nodes and E is a set of directed dependency edges. Each edge $(u, v) \in E$ states that node v may consume only artifacts explicitly produced by u and declared as part of its input surface.

Each node:

$$v = (\mathcal{I}_v, f_v, \mathcal{O}_v) \quad (3)$$

where $\mathcal{I}_v$ denotes the resolved local input state available to node v , f_v is the node-local execution procedure, and $\mathcal{O}_v$ is the structured output artifact selected as the node’s result. Artifacts are not arbitrary transcripts; they are typed outputs with stable boundaries intended for downstream consumption.

This formulation differs from agent loops in two ways. First, the admissible context for each node is declared before execution rather than reconstructed opportunistically from conversational state. Second, node outputs persist as addressable artifacts with explicit lineage, enabling downstream reuse and inspection.

3.1 Design Principles

The execution-lineage model can be summarized by four design principles.

Explicit dependency declaration. A node should not be allowed to consume undeclared upstream state. This principle forces the graph to carry the causal structure of the workflow rather than hiding it in prompt assembly logic.

Local visibility boundaries. Each node should see only the context and dependency artifacts required for its task. Besides improving provenance, this reduces accidental coupling, lowers prompt bloat, and makes it easier to understand why a node produced a given output.

Identity-based reuse. Reuse should be based on proof of equivalence, not similarity of prompts or approximate resemblance of outputs. A runtime should be able to say not merely that two executions “look close enough,” but that they share the same declared structure and resolved inputs.

Deterministic publication. Even when execution involves internal iteration, branching, or validation, the boundary exposed downstream should be stable and canonical. Downstream nodes should depend on a published result, not on whatever happened to be most recently present in a conversational trace.

Taken together, these principles describe a runtime whose main object is not the prompt or the transcript, but the dependency-respecting publication of intermediate computation.

3.2 Intermediate State and Typed Local Boundaries

The core execution object is the intermediate state boundary. In our implementation this is materialized as an artifact, but the deeper claim is executional rather than representational: the system needs a stable, inspectable unit that downstream computation can depend on. An artifact may contain free-form model output, but it is treated by the runtime as a structured unit with an identity, type, and provenance. This matters because the role of an intermediate result is not merely to be read by a human; it is to become a stable dependency surface for later computation.

Each node executes against a typed local state consisting of three classes of entries: immutable context provided at invocation time, immutable dependency artifacts materialized from upstream nodes, and mutable local artifacts produced during the node’s own execution. This separation is important. Without it, a runtime cannot distinguish inherited state from newly produced state, and downstream reasoning about provenance collapses back into transcript reconstruction.

3.3 Dependency Materialization

Dependencies are resolved against actual inputs rather than only logical node names. Concretely, an upstream dependency is identified not only by which node produced it, but by the particular resolved input surface under which that node was executed. This prevents two semantically distinct executions of the same logical node from being aliased together.

Formally, let k_v denote the execution identity of node v . We define:

$$k_v = h(\sigma_v, x_v, \{k_u : u \in \text{pred}(v)\}) \quad (4)$$

where σ_v is the structural specification of node v , x_v is its resolved input hash, and $\text{pred}(v)$ are its immediate predecessors. The hash function h need not be cryptographically special for our argument; what matters is that node identity is a deterministic function of structure, inputs, and upstream identities.

This execution identity provides the basis for cache reuse, replay, and selective invalidation. If k_v is unchanged, prior execution may be reused exactly. If it changes, downstream nodes that depend on k_v are invalidated and re-executed, while unrelated branches remain untouched.

3.4 Execution Semantics

Execution is a forward pass:

$$\mathcal{O}_v = f_v(\mathcal{I}_v) \quad (5)$$

Nodes execute once dependencies are satisfied. At runtime, readiness is decided from the graph rather than inferred by the model. This allows the system to schedule independent branches concurrently while preserving sequential semantics within each node’s local procedure.

In practice, a node may itself contain multiple internal steps, such as context selection, model invocation, validation, or rendering. We treat these as part of f_v . They may be sophisticated, but they remain subordinate to a platform-controlled execution boundary. The model generates content within the node; it does not decide which nodes exist or which undeclared upstream state may be read.

3.5 Determinism

Definition: Execution is deterministic if outputs are reproducible under:

- fixed inputs
- fixed model + parameters
- fixed tool outputs

Under these conditions, determinism means more than “similar answers.” It means that the system can either replay a prior node execution exactly or prove that some aspect of its identity changed and therefore recomputation is required. Determinism is therefore a property of the execution substrate, not a promise that the model itself is universally stable under all deployments.

3.6 Replay and Partial Recomputation

Execution lineage makes replay a first-class runtime mode. When a node’s execution identity matches a previously persisted result, the runtime can restore its artifact and associated execution record rather than regenerate it heuristically. This differs from approximate caching: replay is identity-based and therefore explainable.

Similarly, partial recomputation follows directly from graph structure. If an upstream research artifact changes, only descendants whose execution identities depend on that artifact must be recomputed. The runtime can leave unrelated branches intact. In contrast, prompt-centric systems often collapse all prior work into one evolving context window, making a local change expensive because the system lacks precise invalidation boundaries.

3.7 Invalidation Semantics

The usefulness of an execution graph depends not only on replay, but on *correct* replay refusal. A lineage-aware runtime must be able to explain both why prior work can be reused and why it can no longer be reused. Invalidation therefore becomes a semantic operation over dependency identities rather than a vague sense that “the prompt changed.”

Consider a simple three-stage workflow consisting of retrieval, analysis, and synthesis. If only the synthesis prompt changes, the system should preserve retrieval and analysis while recomputing only synthesis. If a retrieval source changes, analysis and synthesis must be invalidated because their dependency identities are downstream of the changed source. This style of invalidation is routine in build systems and dataflow engines, but remains under-specified in many agent systems, where upstream changes are usually handled by restarting or partially reconstructing a prompt transcript.

This distinction matters operationally. Without explicit invalidation semantics, teams tend to choose between two unsatisfactory behaviors: stale reuse of outputs that no longer match current assumptions, or expensive global reruns that throw away valid prior work. Execution lineage aims to replace that ambiguity with a principled middle ground.

3.8 Canonical Outputs and Observability

An execution system also needs a notion of which output is authoritative at a boundary. In loop-based systems, candidate drafts, self-critiques, tool observations, and final responses are often mixed together in one transcript. A human can often infer which part “counts,” but the runtime itself lacks a reliable canonical object for downstream consumers.

Execution-lineage systems instead select a canonical output at each node boundary. That output may have been produced through multiple internal steps, validations, or revisions, but once selected it becomes the stable downstream dependency surface. This improves observability in two ways. First, it lets operators inspect the exact artifact that

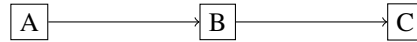
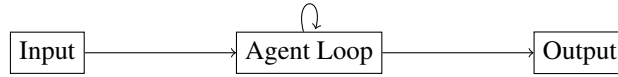
downstream nodes consumed, rather than rereading an entire transcript. Second, it creates a durable execution history in which intermediate candidates, final selected outputs, and dependency identities can be distinguished rather than conflated.

3.9 Execution Lineage vs. Prompt Lineage

Prompt lineage records the history of prompts, messages, and tool observations used during an interaction. It is useful for audit and narration, but it is a poor substrate for composition because it lacks stable execution boundaries. Execution lineage, by contrast, records the graph of artifact-producing computations and the identities that connect them.

The distinction parallels the difference between logs and program structure. Logs are valuable for understanding what happened after the fact; they are not the mechanism by which a system should declare dependencies or reuse work. In our model, reasoning traces may still exist, but they are auxiliary observability surfaces rather than the system’s computational backbone.

4 Execution vs Agent Loop (Diagram)



Top: agent loop. Bottom: execution graph.

5 Research Questions

Our empirical evaluation is designed around lightweight questions that a solo researcher can answer with modest tooling and repeated manual runs rather than large-scale infrastructure.

- **RQ1: Stability.** Does an execution-graph workflow produce less cross-run variation than a prompt-loop workflow when asked to perform the same multi-step task repeatedly?
- **RQ2: Incrementality.** After a small upstream change, does an execution-graph workflow require less rework than a prompt-loop workflow?
- **RQ3: Traceability.** Is it easier to identify what changed and why in an execution-graph workflow than in a prompt-loop workflow?

These questions do not require frontier-scale benchmarking. They are intended to measure systems properties that should be observable even in small, carefully controlled experiments.

6 Methods

6.1 Study Design

We use a within-task comparative design. The same task is executed under two conditions:

- **Prompt-loop baseline:** a conventional chat or agent interaction in which prior work is carried forward through prompts, conversation history, uploaded files, or manually restated context.
- **Execution-graph condition:** the same task represented as explicit stages with declared dependencies and stable intermediate execution boundaries.

The goal is not to compare model intelligence across platforms. Instead, the study isolates the effect of execution substrate. Whenever possible, the same underlying model family, source materials, and task instructions should be used across conditions.

6.2 Task Selection

The experimental task should be simple enough to run manually, but structured enough to expose the differences between global reruns and dependency-aware reruns. A practical template is a three-stage research workflow:

1. **Source collection:** gather a small fixed set of documents, links, or excerpts on a topic.
2. **Analysis:** extract key claims, summarize evidence, and identify tensions or open questions.
3. **Synthesis:** write a short memo or report based on the analysis.

This task can be executed using ordinary web access and a fixed document set. It is intentionally lightweight: one researcher can run it by hand, and the intermediate boundaries are easy to define.

6.3 Experimental Conditions

For the prompt-loop baseline, the researcher runs the workflow in a general-purpose interface such as ChatGPT or Claude. The model is asked to perform the full task end-to-end, with intermediate state preserved only through the session transcript, manually maintained notes, or uploaded files.

For the execution-graph condition, the same workflow is implemented in ThruWire as explicit stages such as *retrieve*, *analyze*, and *write*. Each stage exposes a stable intermediate result and depends only on declared upstream outputs.

To keep the comparison fair and feasible, the experiment should avoid hidden automation advantages. The document set, task prompt, and requested output format should remain as similar as possible across conditions.

6.4 Procedure

Each task instance is run in three phases.

Phase 1: Initial execution Run the full workflow from scratch in both conditions and record:

- wall-clock time to first complete output
- number of user interventions
- availability of identifiable intermediate outputs

Phase 2: Repeat execution Re-run the same task multiple times under fixed inputs and settings. A lightweight but meaningful design is 5–10 repeated runs per condition. Record:

- output similarity across runs
- whether intermediate stages remain stable or drift
- any visible differences in conclusions, structure, or cited evidence

Phase 3: Upstream change After the initial run, introduce a small but meaningful change to an upstream input. For example:

- replace one source document
- modify one source excerpt
- add a short constraint to the analysis stage

Then measure how much work must be repeated to produce an updated final output. In a prompt-loop system this often means reconstructing context and re-running most of the workflow. In an execution graph, the expectation is that only the changed stage and its downstream dependents need to be re-executed.

6.5 Metrics

The metrics are intentionally simple so they can be collected manually or with lightweight scripts.

Stability Measure variation across repeated runs using any of the following low-cost proxies:

- pairwise textual similarity between final outputs
- count of distinct key claims or recommendations across runs
- manual rating of whether final conclusions materially differ

If automated metrics are used, they should be reported as approximate operational proxies rather than as definitive semantic measures.

Incremental update cost Record:

- wall-clock time from upstream edit to updated final output
- number of stages that had to be re-run
- number of manual copy-forward or context-reconstruction actions required

Traceability Traceability is evaluated with a lightweight rubric. For each run, the researcher asks:

- Can I identify which intermediate step produced a given final claim?
- Can I tell which upstream change altered the final output?
- Can I inspect the relevant intermediate state without rereading the full transcript?

These questions can be scored on a 3-point scale (difficult / partial / easy) or summarized descriptively if the paper remains primarily qualitative.

6.6 Implementation Notes for a Solo Researcher

This method is designed to be feasible without extensive infrastructure. It does not require custom benchmarks, large datasets, or many annotators. A single researcher can execute it with:

- one fixed task template
- one small source set per task
- 5–10 repeated runs per condition
- simple timing, note-taking, and optional text-similarity scripts

The strongest version of the experiment is not the largest one. It is the cleanest one: same task, same inputs, small upstream edits, explicit recording of what had to be re-run, and transparent acknowledgment of what is measured manually.

7 Experimental Setup

7.1 Example Task Template

One concrete setup is a short research brief task:

“Using the provided sources, produce a one-page brief summarizing the competitive landscape of X, the strongest evidence for each major claim, and two unresolved questions that require further investigation.”

The task is appropriate because it naturally decomposes into retrieval, analysis, and synthesis, while still being manageable in a manual study.

7.2 Example Upstream Edit

After the initial run, replace one source with a contradictory or higher-quality source and ask the system to update the brief. This produces a controlled intervention that should propagate downstream. The key observation is not whether the answer changes, but how the system changes it: by re-running everything, or by invalidating only the affected downstream stages.

7.3 Controls

To reduce confounds, the researcher should hold constant:

- the source set used in the initial run
- the requested output format
- the high-level task instruction
- the model choice, where platform constraints allow

Any unavoidable differences between platforms should be reported clearly rather than hidden. For example, if one interface does not expose the same model controls or file-handling behavior, that limitation should be stated as part of the experimental context.

8 Results

8.1 Results Template

Table 1 provides a lightweight template for the main comparative results.

Metric	Prompt Loop Baseline	Execution Graph
Cross-run variation	[fill in]	[fill in]
Initial completion time	[fill in]	[fill in]
Update time after upstream edit	[fill in]	[fill in]
Stages re-run after edit	[fill in]	[fill in]
Manual context reconstruction steps	[fill in]	[fill in]
Traceability rating	[fill in]	[fill in]

Table 1: Template for the primary comparison between prompt-loop and execution-graph conditions.

8.2 Expected Outcome Pattern

If the execution-lineage hypothesis is correct, the most important differences should appear in update behavior and traceability rather than in raw one-shot quality alone. Specifically, we expect the execution-graph condition to show:

- lower run-to-run variation for structured multi-step tasks
- fewer stages re-executed after an upstream change
- fewer manual context-reconstruction actions
- clearer identification of which intermediate state caused a final change

Initial completion time may or may not favor the execution graph. In some cases the graph condition may incur modest setup overhead because stages must be declared explicitly. This does not weaken the claim of the paper. The core argument is about reproducibility and incremental operability over time, not minimum latency on a single first run.

8.3 Illustrative Analysis Template

The final writeup can be organized around three observations.

Observation 1: Stability Describe whether repeated runs converged on similar outputs, and whether differences were mostly stylistic or materially substantive.

Observation 2: Incrementality Report how much work was repeated after the upstream edit, emphasizing whether the system supported local invalidation or forced global rerun behavior.

Observation 3: Traceability Describe whether the researcher could directly inspect the intermediate step responsible for a final claim or a downstream change.

8.4 Threats to Validity

This evaluation is intentionally small and should be presented as such. Likely threats include:

- platform differences unrelated to execution model
- limited sample size
- partial reliance on manual scoring
- task-specific results that may not generalize to all workloads

These limitations do not invalidate the experiment. They define its scope. The purpose of the study is to establish a concrete, reproducible small-scale test of execution-lineage claims, not to offer a comprehensive benchmark of all agent systems.

9 Discussion

9.1 Why This Matters

Agent loops optimize for task completion. Execution graphs optimize for:

- persistence
- evolution
- collaboration

This difference becomes more important as AI systems move from one-shot assistants to persistent work environments. In many real settings, the goal is not merely to get an answer once. The goal is to build a body of work that can be revised, extended, audited, and shared across humans and agents. A system that stores only prompts and final answers leaves too much of the work trapped inside transient execution.

Execution lineage changes the unit of improvement. Instead of iterating only on a final output, teams can iterate on the structure that produces outputs. A change to an upstream methodology, research constraint, or transformation rule can propagate through the graph in a principled way. This resembles the transition seen in data engineering, where the object of engineering shifted from isolated scripts to the dependency system that generated downstream tables and reports.

The practical collaboration benefit follows from this, but is not the paper’s main claim. Once intermediate boundaries and dependencies are explicit, humans and agents can intervene at meaningful points rather than by restating instructions into a loop. However, the novelty we emphasize here is the runtime substrate that makes such interventions precise: explicit invalidation boundaries, replayable identities, and deterministic scheduling over a graph.

9.2 Where Execution Graphs Help Most

Execution lineage is unlikely to matter equally for all AI tasks. Its advantages should be most visible in workflows with four properties.

- **Multi-stage structure:** the work naturally decomposes into separable stages such as retrieval, transformation, critique, and synthesis.
- **Revision over time:** upstream assumptions, sources, or constraints are likely to change after an initial run.
- **Shared inspection:** humans or other agents need to inspect intermediate work products directly rather than trust an opaque final answer.
- **Asymmetric recomputation cost:** some stages are expensive or slow enough that global reruns are operationally undesirable.

These properties describe a broad class of realistic AI-native work: research briefs, analysis pipelines, coding workflows, forecasting systems, compliance reviews, and operational runbooks. In such settings, the key question is rarely “can the model produce an answer?” The harder question is whether the system can evolve that answer over time without losing its own structure.

9.3 What This Paper Does Not Claim

The argument here should not be overstated. We do not claim that deterministic graphs replace all useful forms of agentic exploration. Open-ended discovery, broad brainstorming, and ambiguous early-stage problem framing may still benefit from unconstrained loops, reflection, and search. Nor do we claim that explicit execution structure by itself guarantees better one-shot answer quality. A poorly designed graph can still route bad prompts, weak tools, or misleading inputs.

What we claim is narrower and more systems-oriented. Once a workflow has repeated stages, reusable intermediate state, and nontrivial update pressure, leaving its structure implicit becomes a liability. At that point, the graph is not merely one possible representation among many. It becomes a practical mechanism for preserving correctness, reducing unnecessary recomputation, and making the system legible to its operators.

9.4 Limitations

- requires structure upfront
- less flexible for open-ended tasks
- determinism depends on controlled inputs

These limitations are real. Some exploratory tasks are best served by unconstrained loops early in the process, when the system does not yet know what the durable structure should be. Our claim is therefore not that all agentic behavior should disappear, but that once a workflow acquires recurring stages and reusable intermediates, leaving that structure implicit becomes increasingly costly.

In addition, typed artifacts and execution identities introduce design overhead. Someone must define output contracts, dependency boundaries, and invalidation semantics. Poorly chosen boundaries can make a graph brittle or unnecessarily fine-grained. There is also an unresolved question about how best to combine deterministic execution structure with stochastic search methods used in more open-ended planning and creative reasoning.

Finally, determinism is conditional. If upstream APIs, model versions, or external tools change, exact replay may no longer be possible. Execution lineage does not eliminate this reality. It does, however, make the source of change explicit and localizable, which is preferable to a system in which instability is entangled with prompt assembly and hidden context reconstruction.

10 Conclusion

We introduced execution lineage as a replacement for prompt lineage in AI systems. By representing workflows as deterministic graphs of artifact-producing computations, we enable reproducibility, inspectability, and incremental computation.

The broader argument is that many of the failures of current agent systems arise from putting too much system logic inside transient prompt loops. Better models may improve reasoning within those loops, but they do not by themselves create reusable boundaries, durable state, or principled change propagation. Those properties require explicit structure outside the model.

This abstraction aligns AI systems with decades of progress in data and software systems, while adapting those lessons to stochastic model-based computation. We therefore view execution lineage not as an implementation detail, but as a foundational systems layer for persistent AI-native work whose execution must remain inspectable, replayable, and incrementally updatable over time.

References

- [1] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. Understanding the Planning of LLM Agents: A Survey. arXiv:2402.02716, 2024.

- [2] Xinzhe Li. A Review of Prominent Paradigms for LLM-Based Agents: Tool Use (Including RAG), Planning, and Feedback Learning. arXiv:2406.05804, 2024.
- [3] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A Survey on the Memory Mechanism of Large Language Model Based Agents. arXiv:2404.13501, 2024.
- [4] Pengfei Du. Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers. arXiv:2603.07670, 2026.
- [5] Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show Your Work: Scratchpads for Intermediate Computation with Language Models. arXiv:2112.00114, 2021.
- [6] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, 2022.
- [7] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171, 2022.
- [8] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. arXiv:2205.10625, 2022.
- [9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023.
- [10] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023.
- [11] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society. arXiv:2303.17760, 2023.
- [12] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291, 2023.
- [13] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv:2308.00352, 2023.
- [14] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.
- [15] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*, 2023.
- [16] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, 2023.
- [17] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, 2023.
- [18] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. arXiv:2310.04406, 2023.
- [19] Tao Li, Gang Li, Zhiwei Deng, Bryan Wang, and Yang Li. A Zero-Shot Language Agent for Computer Control with Structured Reflection. arXiv:2310.08740, 2023.
- [20] Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. Reinforcement Learning on Web Interfaces Using Workflow-Guided Exploration. arXiv:1802.08802, 2018.
- [21] Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-Language Agent Harnesses. arXiv:2603.25723, 2026.

- [22] Yuxuan Zhang, Haoyang Yu, Lanxiang Hu, Haojian Jin, and Hao Zhang. General Modular Harness for LLM Agents in Multi-Turn Gaming Environments. arXiv:2507.11633, 2025.
- [23] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent Workflow Memory. arXiv:2409.07429, 2024.
- [24] Ruihong Zeng, Jinyuan Fang, Siwei Liu, and Zaiqiao Meng. On the Structural Memory of LLM Agents. arXiv:2412.15266, 2024.
- [25] Shengda Fan, Xin Cong, Yuepeng Fu, Zhong Zhang, Shuyan Zhang, Yuanwei Liu, Yesai Wu, Yankai Lin, Zhiyuan Liu, and Maosong Sun. WorkflowLLM: Enhancing Workflow Orchestration Capability of Large Language Models. arXiv:2411.05451, 2024.
- [26] Dongge Han, Camille Couturier, Daniel Madrigal Diaz, Xuchao Zhang, Victor Rühle, and Saravan Rajmohan. LEGOMem: Modular Procedural Memory for Multi-agent LLM Systems for Workflow Automation. arXiv:2510.04851, 2025.
- [27] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Hinrich Schütze, Volker Tresp, and Yunpu Ma. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. arXiv:2508.19828, 2025.
- [28] Luiz C. Borro, Luiz A. B. Macarini, Gordon Tindall, Michael Montero, and Adam B. Struck. Memori: A Persistent Memory Layer for Efficient, Context-Aware LLM Agents. arXiv:2603.19935, 2026.
- [29] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks. In *EuroSys*, 2007.
- [30] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster Computing with Working Sets. In *USENIX HotCloud*, 2010.
- [31] Apache Software Foundation. Airflow Documentation: DAGs. <https://airflow.apache.org/docs/apache-airflow/3.0.4/core-concepts/dags.html>, accessed May 4, 2026.
- [32] dbt Labs. dbt Developer Hub. <https://docs.getdbt.com/>, accessed May 4, 2026.
- [33] Omar Khattab, Keshav Santhanam, Xiang Lisa Li, Aatmik Gupta, Christopher Potts, and Matei Zaharia. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. arXiv:2310.03714, 2023.
- [34] Leon Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting Is Programming: A Query Language for Large Language Models. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2023.
- [35] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. arXiv:2211.12588, 2022.
- [36] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided Language Models. In *International Conference on Machine Learning*, 2023.
- [37] Lingjiao Chen, Matei Zaharia, and James Zou. How Is ChatGPT’s Behavior Changing over Time? arXiv:2307.09009, 2023.
- [38] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688, 2023.
- [39] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854, 2023.
- [40] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. arXiv:2401.13649, 2024.
- [41] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks? arXiv:2403.07718, 2024.
- [42] Christopher Rawles, Sarah Clinckemaulle, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillcrap, and Oriana Riva. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. arXiv:2405.14573, 2024.

- [43] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972, 2024.
- [44] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. arXiv:2407.18901, 2024.
- [45] Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A Benchmark for General AI Assistants. arXiv:2311.12983, 2023.
- [46] Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, ZhongZhi Li, Yingying Zhang, Le Song, and Qianli Ma. LifelongAgentBench: Evaluating LLM Agents as Lifelong Learners. arXiv:2505.11942, 2025.
- [47] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. arXiv:2507.05257, 2025.
- [48] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory. arXiv:2511.20857, 2025.
- [49] Yiting Shen, Kun Li, Wei Zhou, and Songlin Hu. Mem2ActBench: A Benchmark for Evaluating Long-Term Memory Utilization in Task-Oriented Autonomous Agents. arXiv:2601.19935, 2026.
- [50] Renan Souza, Timothy Poteet, Brian Etz, Daniel Rosendo, Amal Gueroudji, Woong Shin, Prasanna Balaprakash, and Rafael Ferreira da Silva. LLM Agents for Interactive Workflow Provenance: Reference Architecture and Evaluation Methodology. arXiv:2509.13978, 2025.
- [51] Heng Ma, Alexander Brace, Carlo Siebenschuh, Greg Pauloski, Ian Foster, and Arvind Ramanathan. Connecting Large Language Model Agent to High Performance Computing Resource. arXiv:2502.12280, 2025.
- [52] Josh Abramson, Arun Ahuja, Federico Carnevale, Petko Georgiev, Alex Goldin, Alden Hung, Jessica Landon, Timothy Lillicrap, Alistair Muldal, Blake Richards, Adam Santoro, Tamara von Glehn, Greg Wayne, Nathaniel Wong, and Chen Yan. Evaluating Multimodal Interactive Agents. arXiv:2205.13274, 2022.